

Simulação do Sinal no MATLAB

A decisão de gerar os sinais no MATLAB (em vez de tentar simular tudo diretamente no Proteus ou Wokwi) baseia-se na:

- Superação das limitações do Simulador (Wokwi): O Wokwi é excelente para simular a lógica digital do ESP32, mas não possui suporte robusto para fontes de tensão alternada complexas (220V AC). O MATLAB resolve isso criando um "gêmeo digital" matemático do sinal.
- Controle Absoluto de Variáveis (Ground Truth): Se você injeta um sinal real ou simulado de forma analógica, você nunca tem certeza exata de qual é o THD real para comparar com o cálculo do ESP32. Ao criar a onda no MATLAB, você dita as regras: "Eu quero exatamente 20% de 3ª harmônica". Se o ESP32 ler algo diferente, você sabe que o erro está no código C++. E não no sinal.

1. Parâmetros do Sistema e Amostragem

```
f_fundamental = 60; % Frequência esperada da rede elétrica (Hz)
pontos_por_ciclo = 128; % Amostras por ciclo (para a FFT)
amostras = pontos_por_ciclo;
% Eixo do tempo para exatamente 1 ciclo (16.66 ms)
% Remove o último ponto para a onda conectar perfeitamente no loop infinito
t = linspace(0, 1/f_fundamental, amostras + 1);
t(end) = [];
```

Define a base de tempo. Para uma rede de **60Hz**, um ciclo completo leva cerca de **16,66 ms**.

O valor **128** não é aleatório. A Transformada Rápida de Fourier (FFT) que o ESP32 usa exige que o número de amostras seja uma **potência de 2** (Radix-2) para funcionar com máxima eficiência. O código também remove o último ponto temporal **t(end) = []** para garantir que, quando o ESP32 repetir esse vetor em um loop infinito, não haja um "degrau" no formato da onda ao recomear o ciclo.

2. Modelagem das Amplitudes

```
A1 = 311; % Fundamental 60Hz (220V * sqrt(2))
A3 = 60; % 3ª Harmônica (180Hz) - Causa aquecimento no neutro
A5 = 30; % 5ª Harmônica (300Hz)
A7 = 15; % 7ª Harmônica (420Hz)
A_Ruido = 10; % Ruído de Alta Frequência (Ex: 3600Hz de fontes chaveadas)
```

Define os picos de tensão. Como a rede é de **220V RMS**, a amplitude de pico (A1) é matematicamente **311V**. Em seguida, são definidas as amplitudes para os distúrbios (harmônicas ímpares e ruído de alta frequência).

3. Geração dos Sinais (Domínio do Tempo)

```
% Cenário 1: Rede Perfeita
sinal_limpo = A1 * sin(2*pi * f_fundamental * t);
% Cenário 2: Distorção Harmônica Elevada (THD)
sinal_thd = sinal_limpo + ...
    A3 * sin(2*pi * (3*f_fundamental) * t) + ...
    A5 * sin(2*pi * (5*f_fundamental) * t) + ...
    A7 * sin(2*pi * (7*f_fundamental) * t);
```

```
% Cenário 3: Ruído Acoplado (EMI/RFI)
sinal_ruido = sinal_limpo + A_Ruido * sin(2*pi * (60*f_fundamental) * t);
```

Cria três cenários isolados de teste utilizando trigonometria básica. Somando ondas senoidais de frequências múltiplas (180Hz, 300Hz, 420Hz), o código distorce a onda fundamental propositalmente para simular, por exemplo, o efeito de fontes chaveadas ou inversores na rede elétrica.

4. Quantização para o ADC do ESP32

```
% O circuito físico possui offset de 1.65V (Valor 2048 no ADC)
offset_adc = 2048;
% Função anônima para converter Tensão Real -> Leitura ADC com limites
converter_adc = @(sinal) max(min(round(sinal + offset_adc), 4095), 0);
adc_limpo = converter_adc(sinal_limpo);
```

```
adc_thd = converter_adc(sinal_thd);
adc_ruido = converter_adc(sinal_ruido);
```

Esta é a parte mais crítica. O conversor analógico-digital (ADC) do ESP32 tem resolução de 12 bits (lê valores de **0 a 4095**) e queima se receber tensões negativas.

O sinal real de **-311V a +311V** é simulado como se já tivesse passado pelo condicionamento físico: ele recebe um "empurrão" para cima (offset de **2048**, que representa o meio do ADC, ou **1.65V**) e a função **max(min(...))** atua como um grampo de segurança, impedindo que qualquer valor matemático saia da faixa realística do hardware.

5. Exportação Automática de Código C++

```
fprintf("\n// COPIE E COLE OS ARRAYS ABAIXO NO SEU SKETCH.INO (BLOCO 1)\n");
fprintf("// =====\n\n");
% Exporta Sinal Limpo
fprintf('const int sinal_rede_Limpo[%d] = {', amostras);
fprintf('%d, ', adc_limpo(1:end-1)); fprintf('%d;\n', adc_limpo(end));
% Exporta Sinal com THD
fprintf('const int sinal_rede_THD_Alto[%d] = {', amostras);
fprintf('%d, ', adc_thd(1:end-1)); fprintf('%d;\n', adc_thd(end));
% Exporta Sinal com Ruído
fprintf('const int sinal_rede_Ruido_Alto[%d] = {', amostras);
fprintf('%d, ', adc_ruido(1:end-1)); fprintf('%d;\n\n', adc_ruido(end));
```

Automatiza o seu trabalho. Em vez de copiar e colar 128 números formatando as vírgulas na mão, o MATLAB escreve no console a linha exata de código **const int...** pronta para ser colocada no sketch.ino do Wokwi.

6. Validação Matemática da FFT (Ground Truth)

```
% Esta seção prova que a lógica implementada em C++ no ESP32 está correta.
fprintf("// --- VALIDAÇÃO DA FFT (MATLAB vs ESP32) ---\n");
% Janelamento e FFT no sinal com THD
janela = hamming(amostras);
vReal_windowed = sinal_thd .* janela;
vFFT = fft(vReal_windowed);
% Conversão para Magnitude e Normalização
vMag = abs(vFFT) * (2 / amostras);
% Extração dos Bins de interesse (No MATLAB o índice começa em 1, logo Bin[1] é idx 2)
ampFund = vMag(2); % 60Hz
ampH3 = vMag(4); % 180Hz
ampH5 = vMag(6); % 300Hz
ampH7 = vMag(8); % 420Hz
% Cálculo do THD
thd_calculado = (sqrt(ampH3^2 + ampH5^2 + ampH7^2) / ampFund) * 100;
fprintf("// Fundamental (60Hz): %.2f V\n", ampFund);
fprintf("// THD Calculado: %.2f %%\n", thd_calculado);
if thd_calculado > 5.0
    fprintf("// DIAGNÓSTICO: ALERTA (THD > 5%%)\n");
end
```

Roda o mesmo processamento digital que o ESP32 fará. O MATLAB aplica a janela de Hamming, executa a FFT nativa, normaliza as grandezas e calcula o THD em porcentagem.

Servi para comparação de resultados. O MATLAB imprimirá, por exemplo, que o THD desse sinal distorcido é de **22,34%**. Quando você rodar o ESP32 no Wokwi, ele deve imprimir um valor idêntico (ou extremamente próximo). Se os valores baterem, o algoritmo embarcado está matematicamente provado.

7. Visualização Gráfica

```
figure('Name', 'Análise de Sinais Gerados', 'Position', [100, 100, 900, 600]);
% Gráfico 1: Sinais no Tempo (Visão do Osciloscópio)
subplot(2,1,1);
hold on;
plot(t*1000, sinal_limpo, 'g', 'LineWidth', 2);
plot(t*1000, sinal_thd, 'r', 'LineWidth', 1.5);
plot(t*1000, sinal_ruido, 'b', 'LineWidth', 1);
hold off;
title('Sinais de Energia Simulados (Domínio do Tempo)');
xlabel('Tempo (ms)'); ylabel('Tensão (V)');
legend('Sinal Limpo', 'Sinal com THD', 'Sinal com Ruído EMI');
grid on;
```

% Gráfico 2: Espectro de Frequência do Sinal Distorcido (Visão da FFT)

```
subplot(2,1,2);
```

```
bins = 0:(amostras/2);
```

```
stem(bins, vMag(1:amostras/2 + 1), 'filled', 'MarkerFaceColor', 'm', 'Color', 'm');
```

```
title('Espectro de Magnitude da FFT (Sinal com THD)');
```

```
xlabel('Bins de Frequência'); ylabel('Amplitude (V)');
```

```
xlim([0 15]); % Limita a visão aos primeiros 15 harmônicos igual a interface do projeto
```

```
xticks(0:15);
```

```
grid on;
```

Gera os dois gráficos (no domínio do tempo e no domínio da frequência). Serve para visualizar o que acabou de gerar. O **stem** cria o característico gráfico de "barras" do espectro de magnitude, mostrando exatamente onde a energia de cada distúrbio está localizada.